

Operating Systems

Project Phase 3: Multithreaded Remote Shell Server

NYU Abu Dhabi

Shota Matsumoto
sm11745@nyu.edu

Sihyun Kim
sk11019@nyu.edu

April 17, 2026

Contents

1	Architecture and Design	3
1.1	High-Level Overview	3
1.2	File Structure	3
1.3	Key Design Decisions	3
2	Implementation Highlights	3
2.1	Thread Creation and Detachment	3
2.2	Race Condition Prevention	4
2.3	Output Capture	4
2.4	EOF Sentinel for Reliable Receive	4
2.5	Error Handling	5
3	Execution Instructions	5
3.1	Compilation	5
3.2	Running	5
3.3	Cleanup	5
4	Testing	6
4.1	Test 1: Compilation and Three Simultaneous Client Connections	6
4.2	Test 2: Concurrent Command Execution	6
4.3	Test 3: Client Disconnect via <code>exit</code>	8
4.4	Test 4: Error Handling — Invalid Command	9
5	Challenges	10
6	Division of Tasks	10
7	References	10

1 Architecture and Design

1.1 High-Level Overview

Phase 3 extends the Phase 2 TCP remote shell server to support multiple simultaneous clients using POSIX threads (`pthread`s). The system consists of three main components: a multithreaded server (`server.c`), a client (`client.c`), and the shared shell logic from Phase 1 (`parser.c`, `executor.c`, `common.h`).

1.2 File Structure

File	Role
<code>server.c</code>	Multithreaded TCP server, one thread per client
<code>client.c</code>	TCP client with sentinel-based receive loop
<code>parser.c</code>	Tokenizes and parses shell command lines
<code>executor.c</code>	Executes parsed commands with pipes and redirections
<code>common.h</code>	Shared structs (<code>Command</code> , <code>ExecutionPlan</code>) and function declarations
<code>main.c</code>	Local interactive shell (Phase 1)
<code>Makefile</code>	Separate compilation for all targets

1.3 Key Design Decisions

One thread per client. Each accepted connection is handed off to a new detached `pthread`. Detached threads automatically release their resources when they finish, avoiding the need for `pthread_join`.

Mutex for client numbering. A `pthread_mutex_t` protects the global client counter so that two threads cannot be assigned the same client number in a race condition.

EOF sentinel (`\x04`). The server appends the ASCII EOF character (`0x04`) after every response. The client reads in a loop until it detects this sentinel, solving the problem of not knowing when a response is fully received — especially for output that is exactly `BUFFER_SIZE` bytes.

Output capture via pipe + fork. `execute_and_capture()` forks a child process, redirects its `stdout` and `stderr` to a pipe, then reads the output back in the parent so it can be sent over the socket.

2 Implementation Highlights

2.1 Thread Creation and Detachment

When a client connects, the server spawns a dedicated thread using `pthread_create` with the `PTHREAD_CREATE_DETACHED` attribute. This means the OS automatically reclaims thread resources on exit.

```

1 pthread_attr_t attributes;
2 pthread_attr_init(&attributes);
3 pthread_attr_setdetachstate(&attributes, PTHREAD_CREATE_DETACHED);
4 pthread_create(&tid, &attributes, handling_client, client_arg);
5 pthread_attr_destroy(&attributes);

```

2.2 Race Condition Prevention

A mutex ensures each client receives a unique, sequential ID even when multiple connections arrive nearly simultaneously:

```

1 pthread_mutex_lock(&client_count_mutex);
2 global_num_client++;
3 int cur_client_num = global_num_client;
4 pthread_mutex_unlock(&client_count_mutex);

```

2.3 Output Capture

The `execute_and_capture()` function creates a pipe, forks a child that runs `execute_plan()`, and reads all output back into a dynamically growing buffer:

```

1 pipe(pipefd);
2 pid = fork();
3 if (pid == 0) {
4     dup2(pipefd[1], STDOUT_FILENO);
5     dup2(pipefd[1], STDERR_FILENO);
6     execute_plan(plan);
7     exit(0);
8 }
9 // parent reads from pipefd[0] into buffer

```

2.4 EOF Sentinel for Reliable Receive

The server appends `\x04` after every response so the client knows when the full output has arrived:

```

1 // server side
2 send(client_fd, output, strlen(output), 0);
3 send(client_fd, "\x04", 1, 0); // EOF sentinel

```

```

1 // client side
2 char *eof_pos = memchr(response, '\x04', num_bytes_received);
3 if (eof_pos) {
4     *eof_pos = '\0';
5     if (eof_pos > response) printf("%s", response);
6     break;

```

```
7 }
```

2.5 Error Handling

- `fork()`, `pipe()`, `socket()`, `bind()`, `listen()`, `accept()`, `pthread_create()` failures are all checked with `perror()` and appropriate fallback behavior.
- If `parse_line()` returns `NULL`, an error message is sent directly to the client.
- Unknown commands are detected by scanning the output for keywords such as "not found", "No such file", and "Error", and are logged with the `[ERROR]` tag on the server.
- Empty commands produce a newline followed by the sentinel so the client is never stuck on `recv()`.

3 Execution Instructions

3.1 Compilation

```
1 make
```

This produces three executables: `myshell` (local shell), `server`, and `client`.

3.2 Running

Open at least two terminals (or SSH sessions on the remote server).

Terminal 1 — start the server:

```
1 ./server
```

Terminal 2, 3, 4 — start clients:

```
1 ./client          # connects to localhost by default
2 ./client 192.168.x.x # or specify a remote IP
```

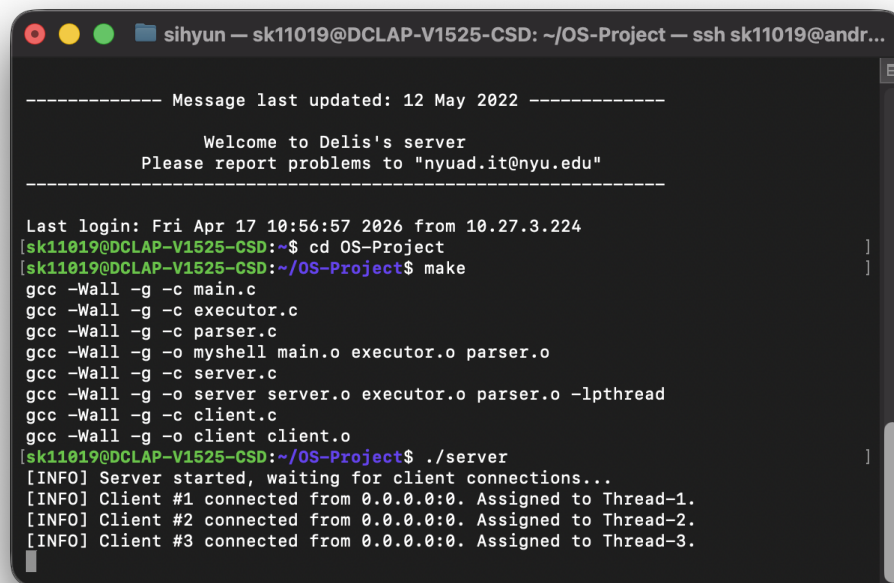
3.3 Cleanup

```
1 make clean
```

4 Testing

4.1 Test 1: Compilation and Three Simultaneous Client Connections

The project was compiled using `make` on the remote Linux server. After starting the server with `./server`, three clients were connected simultaneously from separate SSH terminals. The server correctly printed `[INFO] Client #N connected from 0.0.0.0:0. Assigned to Thread-N` for each client, confirming that multithreaded client handling works correctly.



```
----- Message last updated: 12 May 2022 -----
Welcome to Delis's server
Please report problems to "nyuad.it@nyu.edu"
-----

Last login: Fri Apr 17 10:56:57 2026 from 10.27.3.224
[sk11019@DCLAP-V1525-CSD:~$ cd OS-Project
[sk11019@DCLAP-V1525-CSD:~/OS-Project$ make
gcc -Wall -g -c main.c
gcc -Wall -g -c executor.c
gcc -Wall -g -c parser.c
gcc -Wall -g -o myshell main.o executor.o parser.o
gcc -Wall -g -c server.c
gcc -Wall -g -o server server.o executor.o parser.o -lpthread
gcc -Wall -g -c client.c
gcc -Wall -g -o client client.o
[sk11019@DCLAP-V1525-CSD:~/OS-Project$ ./server
[INFO] Server started, waiting for client connections...
[INFO] Client #1 connected from 0.0.0.0:0. Assigned to Thread-1.
[INFO] Client #2 connected from 0.0.0.0:0. Assigned to Thread-2.
[INFO] Client #3 connected from 0.0.0.0:0. Assigned to Thread-3.
```

Figure 1: Server terminal: successful `make` compilation and three simultaneous client connections assigned to separate threads

4.2 Test 2: Concurrent Command Execution

Three clients sent different commands simultaneously:

- Client #1: `ls -l`
- Client #2: `unknowncmd` (invalid command — error case)
- Client #3: `pwd`

The server correctly logged `[RECEIVED]`, `[EXECUTING]`, and `[OUTPUT]/[ERROR]` tags for each client in an interleaved manner, confirming true concurrent execution across threads.

```
sihyun — sk11019@DCLAP-V1525-CSD: ~/OS-Project — ssh sk11019@andr...

[sk11019@DCLAP-V1525-CSD:~/OS-Project$ make
gcc -Wall -g -c main.c
gcc -Wall -g -c executor.c
gcc -Wall -g -c parser.c
gcc -Wall -g -o myshell main.o executor.o parser.o
gcc -Wall -g -c server.c
gcc -Wall -g -o server server.o executor.o parser.o -lpthread
gcc -Wall -g -c client.c
gcc -Wall -g -o client client.o
[sk11019@DCLAP-V1525-CSD:~/OS-Project$ ./server
[INFO] Server started, waiting for client connections...
[INFO] Client #1 connected from 0.0.0.0:0. Assigned to Thread-1.
[INFO] Client #2 connected from 0.0.0.0:0. Assigned to Thread-2.
[INFO] Client #3 connected from 0.0.0.0:0. Assigned to Thread-3.
[RECEIVED] [Client #1 - 0.0.0.0:0] Received command: "ls -l"
[EXECUTING] [Client #1 - 0.0.0.0:0] Executing command: "ls -l"
[OUTPUT] [Client #1 - 0.0.0.0:0] Sending output to client:
total 1260
-rw-rw-r-- 1 sk11019 sk11019      974 Apr 17 11:00 Makefile
drwxrwxr-x 2 sk11019 sk11019    4096 Apr 17 11:00 OS Screenshots for Project 2
-rw-rw-r-- 1 sk11019 sk11019 1036381 Apr 17 11:00 Operating_System_Project_Phase
_2.pdf
-rwxrwxr-x 1 sk11019 sk11019   21712 Apr 17 11:05 client
-rw-rw-r-- 1 sk11019 sk11019   4244 Apr 17 11:00 client.c
-rw-rw-r-- 1 sk11019 sk11019  13672 Apr 17 11:05 client.o
-rw-rw-r-- 1 sk11019 sk11019    949 Apr 17 11:00 common.h
-rw-rw-r-- 1 sk11019 sk11019   4284 Apr 17 11:00 executor.c
-rw-rw-r-- 1 sk11019 sk11019  13312 Apr 17 11:05 executor.o
-rw-rw-r-- 1 sk11019 sk11019   1299 Apr 17 11:00 main.c
-rw-rw-r-- 1 sk11019 sk11019   9096 Apr 17 11:05 main.o
-rwxrwxr-x 1 sk11019 sk11019  31848 Apr 17 11:05 myshell
-rw-rw-r-- 1 sk11019 sk11019   8932 Apr 17 11:00 parser.c
-rw-rw-r-- 1 sk11019 sk11019  15768 Apr 17 11:05 parser.o
-rwxrwxr-x 1 sk11019 sk11019  45520 Apr 17 11:05 server
-rw-rw-r-- 1 sk11019 sk11019  10499 Apr 17 11:00 server.c
-rw-rw-r-- 1 sk11019 sk11019  24136 Apr 17 11:05 server.o

[RECEIVED] [Client #2 - 0.0.0.0:0] Received command: "unknowncmd"
[EXECUTING] [Client #2 - 0.0.0.0:0] Executing command: "unknowncmd"
[ERROR] [Client #2 - 0.0.0.0:0] Command not found: "unknowncmd"
[OUTPUT] [Client #2 - 0.0.0.0:0] Sending error message to client: "Command not f
ound: unknowncmd
"
[RECEIVED] [Client #3 - 0.0.0.0:0] Received command: "pwd"
[EXECUTING] [Client #3 - 0.0.0.0:0] Executing command: "pwd"
[OUTPUT] [Client #3 - 0.0.0.0:0] Sending output to client:
/home/sk11019/OS-Project

[RECEIVED] [Client #3 - 0.0.0.0:0] Received command: "exit"
[INFO] [Client #3 - 0.0.0.0:0] Client requested disconnect. Closing connection.
[INFO] Client #3 disconnected.
```

Figure 2: Server log showing interleaved [RECEIVED], [EXECUTING], and [OUTPUT] / [ERROR] messages from three concurrent clients

```
sihyun — sk11019@DCLAP-V1525-CSD: ~/OS-Project — ssh sk11019@andr...
[sk11019@DCLAP-V1525-CSD:~/OS-Project$ ./client
$ ls -l
total 1260
-rw-rw-r-- 1 sk11019 sk11019 974 Apr 17 11:00 Makefile
drwxrwxr-x 2 sk11019 sk11019 4096 Apr 17 11:00 OS Screenshots for Project 2
-rw-rw-r-- 1 sk11019 sk11019 1036381 Apr 17 11:00 Operating_System_Project_Phase_2.pdf
-rwxrwxr-x 1 sk11019 sk11019 21712 Apr 17 11:05 client
-rw-rw-r-- 1 sk11019 sk11019 4244 Apr 17 11:00 client.c
-rw-rw-r-- 1 sk11019 sk11019 13672 Apr 17 11:05 client.o
-rw-rw-r-- 1 sk11019 sk11019 949 Apr 17 11:00 common.h
-rw-rw-r-- 1 sk11019 sk11019 4284 Apr 17 11:00 executor.c
-rw-rw-r-- 1 sk11019 sk11019 13312 Apr 17 11:05 executor.o
-rw-rw-r-- 1 sk11019 sk11019 1299 Apr 17 11:00 main.c
-rw-rw-r-- 1 sk11019 sk11019 9096 Apr 17 11:05 main.o
-rwxrwxr-x 1 sk11019 sk11019 31848 Apr 17 11:05 myshell
-rw-rw-r-- 1 sk11019 sk11019 8932 Apr 17 11:00 parser.c
-rw-rw-r-- 1 sk11019 sk11019 15768 Apr 17 11:05 parser.o
-rwxrwxr-x 1 sk11019 sk11019 45520 Apr 17 11:05 server
-rw-rw-r-- 1 sk11019 sk11019 10499 Apr 17 11:00 server.c
-rw-rw-r-- 1 sk11019 sk11019 24136 Apr 17 11:05 server.o
$ exit
Disconnected from server.
sk11019@DCLAP-V1525-CSD:~/OS-Project$
```

Figure 3: Client #1 terminal: `ls -l` output correctly received from server

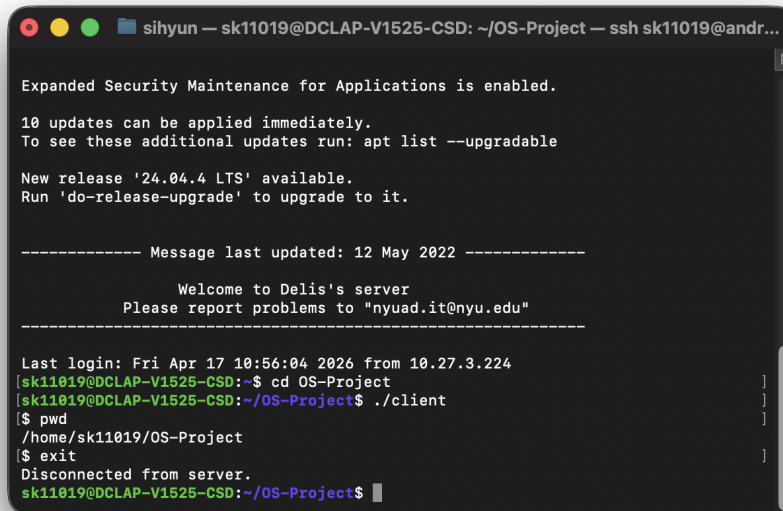
```
sihyun — sk11019@DCLAP-V1525-CSD: ~/OS-Project — ssh sk11019@andr...
https://ubuntu.com/engage/secure-kubernetes-at-the-edge
Expanded Security Maintenance for Applications is enabled.
10 updates can be applied immediately.
To see these additional updates run: apt list --upgradable
New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.
----- Message last updated: 12 May 2022 -----
Welcome to Delis's server
Please report problems to "nyuad.it@nyu.edu"
-----
Last login: Fri Apr 17 10:56:22 2026 from 10.27.3.224
[sk11019@DCLAP-V1525-CSD:~$ cd OS-Project
[sk11019@DCLAP-V1525-CSD:~/OS-Project$ ./client
$ unknowncmd
Command not found: unknowncmd
$
```

Figure 4: Client #2 terminal: Command not found: `unkowncmd` error correctly received from server

4.3 Test 3: Client Disconnect via exit

Client #3 typed `exit`. The client displayed `Disconnected from server.` and terminated cleanly. The server logged `[INFO] Client #3 requested disconnect. Closing`

connection. followed by [INFO] Client #3 disconnected. The remaining clients continued operating without interruption, confirming that thread isolation is working correctly.



```
Expanded Security Maintenance for Applications is enabled.

10 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

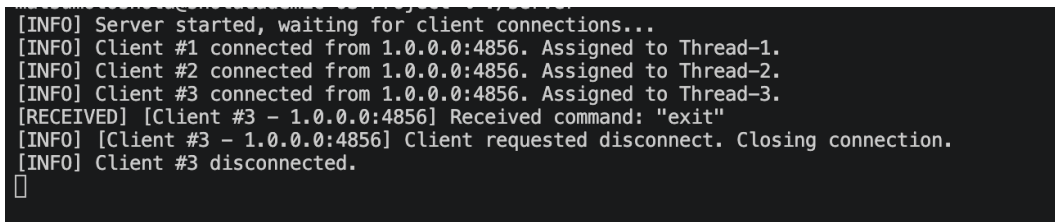
New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

----- Message last updated: 12 May 2022 -----

Welcome to Delis's server
Please report problems to "nyuad.it@nyu.edu"
-----

Last login: Fri Apr 17 10:56:04 2026 from 10.27.3.224
[sk11019@DCLAP-V1525-CSD:~$ cd OS-Project
[sk11019@DCLAP-V1525-CSD:~/OS-Project$ ./client
|$ pwd
/home/sk11019/OS-Project
|$ exit
Disconnected from server.
sk11019@DCLAP-V1525-CSD:~/OS-Project$
```

Figure 5: Client #3 terminal: `pwd` output followed by clean disconnect via `exit`



```
[INFO] Server started, waiting for client connections...
[INFO] Client #1 connected from 1.0.0.0:4856. Assigned to Thread-1.
[INFO] Client #2 connected from 1.0.0.0:4856. Assigned to Thread-2.
[INFO] Client #3 connected from 1.0.0.0:4856. Assigned to Thread-3.
[RECEIVED] [Client #3 - 1.0.0.0:4856] Received command: "exit"
[INFO] [Client #3 - 1.0.0.0:4856] Client requested disconnect. Closing connection.
[INFO] Client #3 disconnected.
```

Figure 6: Server terminal: [INFO] Client #3 requested disconnect. Closing connection. and [INFO] Client #3 disconnected. confirming graceful disconnect

4.4 Test 4: Error Handling — Invalid Command

An invalid command `unknowncmd` was sent from Client #2. The server detected the failure, logged [ERROR] with the client label, and sent the error message back to the client. The client correctly displayed `Command not found: unknowncmd`, confirming that error propagation from server to client works as expected.

The client-side output for this test is visible in Figure 4 (Test 2, Client #2).

5 Challenges

Race condition on client numbering (server.c). Initially, two threads connecting at nearly the same time could increment the global client counter simultaneously and receive the same ID. We solved it by wrapping the counter increment in a `pthread_mutex_lock/unlock` block so only one thread can modify the counter at a time.

Client hanging on `recv()` (client.c). The original receive loop in `client.c` terminated when `num_bytes_received < BUFFER_SIZE - 1`, which failed for responses that were exactly `BUFFER_SIZE - 1` bytes, which caused the client to hang indefinitely waiting for more data. We fixed it by having the server append an EOF sentinel (`\x04`) after every response, and updating the client to detect it using `memchr()` and break out of the loop immediately.

Capturing child process output. Since `execute_plan()` writes to `stdout` directly, capturing its output required an extra `fork() + pipe()` layer in `execute_and_capture()` to redirect and collect the output before sending it over the socket.

6 Division of Tasks

Task	Member
Multithreaded server (<code>server.c</code>)	Shota
Client sentinel fix (<code>client.c</code>)	Sihyun
Testing and screenshots	Sihyun
Report	Both

7 References

- Course Lecture and Lab Slides, *Operating Systems*, NYU Abu Dhabi, Spring 2026.
- Anthropic, *Claude AI Assistant*, claude.ai, 2026. Used for debugging assistance and code review during development.